H-264/RTSP Multicast Stream Integrity

Giuseppe Cattaneo[1](✉), Andrea Bruno[1], and Fabio Petagna[2]

[1] Dipartimento di Informatica, Università degli Studi di Salerno, Fisciano 84084, SA, Italy
{cattaneo,andbruno}@unisa.it

[2] eTuitus, Spinoff dell'Università degli Studi di Salerno, Fisciano 84084, SA, Italy
fabio.petagna@etuitus.it

Abstract. In this paper we discuss about stream integrity problem and specifically we propose a method to protect a live video stream against forgery. In particular we propose to use cryptographic techniques to provide integrity and source identity embedding in the source stream a hash chain built on the top of each frame as produced by the source.

We first present a real life scenario in which stream integrity could be considered an enabler, then we describe our method to perform real time integrity check.

In last section an implementation is sketched to show main implementation parameter.

Keywords: Live video stream · Video integrity · Stream integrity · Digital forensics · H.264 · RTSP

1 Introduction

Live video streaming is a useful way to reduce the distances and enable participation to virtual meeting saving transfer time and therefore earth pollution.

A typical scenario that can benefit of such tool is the remote testimony in a trial. For example in case of testimony of a witness included in a witness relocation program for his own safety, the jury have to hear the testimony, but is risky and expensive to move the testimony. In this case a system to permit remote testimony could be valuable. Unfortunately actual video conference systems suffer of many vulnerability that made these tool unreliable and therefore unusable in legal context. In fact standard video-conference use does not allow endpoint authentication, end-to-end encryption and integrity check (non repudiation).

Moreover the scenario just described in many cases requires multicast communication to make the source video stream available to several destination. Again multicast protocols do not allow standard protocol for privacy like Transport Layer Security (TLS) and therefore are subject to man in the middle attack. As for the author knowledge is literature is not defined a secure multicast protocol.

© Springer International Publishing AG 2017
S. Battiato et al. (Eds.): ICIAP 2017, Part II, LNCS 10485, pp. 558–568, 2017.
https://doi.org/10.1007/978-3-319-68548-9_51

In this paper we present a general method to secure a multicast live video stream. It has been implemented for demo purposes for H.264 stream. Our method takes a standard H.264 stream as input and produce a secure stream that holds the following properties:

P.1 *Integrity check:* The live video stream cannot be modified without been noticed by the client. A stream can be altered using one or more of the following operations: (a) frame content modification, (b) frame injection or deletion.

P.2 *Source authentication:* The source identity can be certified and verified by all the destination endpoints.

P.3 *Real time:* The verification procedure must be available directly on the received stream without introducing significant delay. This is opposite to verification procedure that are performed on the closed file where the stream has been saved.

P.4 *Multicast:* The method must apply also in one-to-many communication protocols. This is opposite to end-to-end security protocols.

The first two properties holds also after that the stream has been stored in a file. As side effect of first properties in a saved stream the frame order is preserved.

In digital forensics area many solution have been proposed for preserving the video acquired as digital evidence [17], for source camera identification [5,10–12] and for video forgery detection [6,9,15], but on the contrary in literature there are few examples of methods to secure real-time streaming [8,16,20,22].

After the method has been designed we implemented a prototype in a real life context to verify the impact produced by our solution. We adopted the H.264 container over Real-time Streaming Protocol (RTSP) because of it wide spread. The RTSP uses the Secure Sockets Layer (SSL) to provide the first three properties, but unfortunately this can be used only on Point-to-Point communications.

In short we propose to add a nonce negotiation during `DESCRIBE` phase of RTSP to Session Description Protocol (SDP) file in addition to the other connection parameter. This nonce will be used to start a hash chain that will allow to check the frame sequence integrity. In addition, periodically, the camera will sign the current element of the hash chain in order to provide property Sect. 1.

We also observed that because RTSP over UDP is a lossy protocol we have proposed a recovery mechanism that can be applied when the number of lost packet is under a fixed threshold.

Finally as further specification we required that the stream encoded by our method should be decoded and viewed with a standard RTSP\H.264 player unaware of such protocol. In other word the protocol should be transparent to the source stream. In the previous scenario this may be the case of the journalist or the audience that want to assist to the testimony but do not interested in checking the stream integrity.

Structure of the Paper: In Sect. 2 we analyse the current state of the art for the problem of stream integrity, and in Sect. 3 we give an overview of the H.264

and RTSP. In Sect. 4 we describe detail our solution and in Sect. 5 we introduce some implementation details to show how the method can be embedded in a real life protocol and the issue we had to face. Finally in Sect. 6 we show our conclusions and some propositions for future works.

2 State of the Art

The topic of video integrity in the field of digital forensics has been extensively discussed in scientific literature, and it mainly concerns the identification of forgery produced by manipulating a video file. In this paper we want focus on the possibility to check the integrity of a live stream in real-time. This specification rules out all the traditional tools that works on saved files.

When the authorities seize a video to be used as evidence in a trial, they have to proof that it has not been tampered in any way, and possibly they have also to identify its source. To identify the source of the video, is possible to use techniques of *Source Camera Identification* like the largely used Pixel Non Uniformity (PNU) proposed by *Fridrich et al.* [11] for digital images and successively been extended to digital video also as proposed by the same authors in [10]. Here the authors show a technique to determine if two videos have been taken by the same camera, in particular they use a normalized cross-correlation on the input noise from the input videos.

Hsu et al. [13] proposed a refinement of this technique by defining the concept of *temporal sensor pattern noise* of a video, with respect to a reference camera, and the correlation is performed using a Gaussian Mixture Model (GMM).

Byram et al. [6] presents a technique to detect duplicate and modified copies of a specific video based only on the image sensor characteristics. The authors in [14] present a forgery detection system specifically designed for surveillance videos [3,7]. They analyze the peculiar characteristics of these videos and then they propose to transform the SPN for each video by applying a *Minimum Average Correlation Energy* (MACE) filter to identify both RGB and infrared video. The manipulation are detected by estimating the scaling factor and calculating the correlation coefficient.

Cattaneo et al. [9] analyzed the possibility to use the PNU to detect the insertion of *alien frames* in a video, when the injected frames have been recorded with a different camera.

Ramaswamy and Rao proposed [16] a method for digital authentication of video sequence compressed using H.264/Advanced Video Codec (AVC) by using traditional digital signature. This method was intended for signing video after it has been saved in a file to guarantee that cannot be applied any further modification to it.

All the systems analysed so far works off-line on a video after it has been saved in a file. Therefore cannot be applied to our scenario.

Among the on-line systems, privacy has been enhanced by [20] where the authors proposed an approach based on the state-of-art symmetric cypher to realize a H.264/AVC and SVC compliant encrypted stream. Even if compliant with standard the stream could not be read by a standard player due to encryption.

3 Operating Environment

In the next section we will give in short an overview of the H.264/AVC/SVC Sect. 3.1 and RTSP/RTP Sect. 3.2 useful for the rest of our work.

Next we introduce the properties of cryptographic function Sect. 3.3 used in our method Sect. 4.

3.1 H.264/AVC/SVC

H.264/AVC was proposed as a standard by ITU-T since 2003 [21] and recently moved to version 11 of the standard [1].

Basically H.264/AVC is similar to previous video coding standards.Those are the basic steps for coding:

1. The picture is subdivided into macroblocks of 16×16 blocks of luma samples
2. Inter or intra prediction (B-frames or P-frames)
3. Transformation and quantization
4. Entropy coding
5. Network Abstraction Layer (NAL) unit assembly

In 2007 [2] was introduced with the the annex G the Scalable Video Codec (SVC), the standardization of an high quality video bitstream that also contains one or more subset bitstreams.

H.264/AVC has a clear structure which distinguishes a coding layer (Video Coding Layer (VCL) and non-VCL) and a NAL. The first one is responsible for the coded representation of the pictures in the stream, while the second formats this representation and provides additional header information.

The NAL units can contain different unit types, and if a player is unable to understand a (NUT) it can skip it. The NAL type from 24 to 31 are unspecified and are available for used defined types.

3.2 RTSP/RTP in Short

RTSP is a network control protocol for real-time streaming frequently used with Real-time Transport Protocol (RTP) as transport protocol which generally runs over User Datagram Protocol (UDP). RTSP defines a set of control sequences useful in controlling multimedia playback.

For the need of this work we show only the control sequences will be necessary in the following Sects. 4 and 5, more details can be found in RFC 2326 [19].

`DESCRIBE` This control sequence is used to ask the server the presentation description typically in SDP format.

`PLAY` This control sequence is used to start all the media streams.

`TEARDOWN` This control sequence is used to terminate all the media streams.

3.3 Cryptographic Functions

Cryptographic Hash Function. A cryptographic hash function is a special class of hash function that has certain properties which make it suitable for use in cryptography. The ideal cryptographic hash function as presented in [18] has five main properties:

- it is deterministic so the same message always results in the same hash
- it is light to compute the hash value for any given message
- it is infeasible to generate a message from its hash value except by trying all possible messages
- any change to a message, even one single bit, must produce a new completely different hash value without any apparent correlation with the old hash value
- it is infeasible to find two different messages with the same hash value.

Digital Signature. A digital signature is a mathematical scheme for demonstrating the authenticity of digital messages or documents. To be valid, digital signatures require the following properties as presented in [18]:

- Authenticity: a valid signature implies that the signer deliberately signed the associated message
- Unforgeability: only the signer can give a valid signature for the associated message
- Non-re-usability: the signature of a document can not be used on another document
- Non-repudiation: the signer can not deny having signed a document that has valid signature
- Integrity: ensure the contents have not been modified.

4 Our Method

In this section we describe in detail our solution to check the integrity of live multicast video stream.

In particular our method can be split in three procedures: first of all the setup procedure Sect. 4.1, in which all the participants negotiate the parameters used during the rest of the process; the sender procedure Sect. 4.2 in which the sender builds an hash chain with video frame digests, and on a time base digitally signs some of these hash values; the receiver procedure Sect. 4.3 in which on the clients the hash chain and the signature carried on by the stream are verified.

At the end we show also the recovery procedure Sect. 4.4 that allows the client to restart the method when some RTSP packets are lost.

4.1 Setup Procedure

This procedure uses the standard SDP like in a traditional RTSP session. Before starting the streaming session with the Secure Remote Camera (SRC) C the

procedure share with all the clients a *nonce* value F_0 and its public key Pk_C. The *nonce* value is an arbitrary number that may only be used once which identifies the session and will be used as the starting value F_0 of the hash chain in the next procedure Sect. 4.2. The public key Pk_C is included in a standard $X.509$ certificate released by an official certification authority to uniquely identify the source. The corresponding private key Sk_C is stored on the server in a secure device.

4.2 Sender Procedure

This procedure is executed on the server-side and, for seek of clarity: can be divided in two sub procedure, (1) *Frame order integrity* and (2) *Source authentication*.

Frame Order Integrity. According to property (P.1) in the Sect. 1, in order to enable stream integrity checking, we implement a hash chain, i.e. we concatenate each frame of the stream with its successor. In facts the hash value for the I-frame F_i is computed, as shown in (1), adding the hash value of the previous I-frame F_{i-1} to the content of the I-frame F_i (HE_{i-1}). The first element of the chain is the hash of the element F_0 generated by setup procedure Sect. 4.1.

$$HE_i = \begin{cases} Hash(F_0) & \text{if } i = 0 \\ Hash(HE_{i-1} + F_i) & \text{if } i > 0 \end{cases} \tag{1}$$

If a malicious user would delete one or more I-frames the chain is broken and the hash value cannot be recomputed (as the hash function cannot be inverted). Analogously if one or more I-frames are injected in the stream the I-frame produced after the injection will report an invalid hash value in the chain.

Therefore the hash chain creates a verifiable and ordered cryptographic link between the current and the already issued I-frames.

Each value HE_i is sent to the RTSP in the NAL with one of the unused NUT along with the content of the corresponding frame F_i.

Source Authentication. Another property of the secure stream is the *Source Authentication*. In every moment the receivers must be able to verify the identity of the sender.

To achieve this goal we use an asymmetric digital signature. Given n, a parameter statically defined on the SRC or adaptively calculated depending the camera CPU load and the network load, every n I-frames in the stream the digital signature DS_{HE_i} of the value HE_i is calculated for the current I-frame F_i using the private key Sk_S. The resulting value is added to the RTSP stream in the NAL with one of the unused NUT in the first available packet. In order to avoid any delay to the input string the value of the digital signature is sent asynchronously with respect to the corresponding frame F_i.

$$DS_{HE_i} = Sign(HE_i, Sk_S) \tag{2}$$

H.264 clients have been designed to skip the entire NAL unit when an unspecified NUT is received. That is why we used this way to send extra data added by procedure Sect. 4.2. As consequence a client that is not aware of our method will be anyway able to process the stream and decode the video.

4.3 Receiver Procedure

This procedure is executed on client-side and is in charge of performing on-line integrity check on each received I-frame. Moreover the procedure must verify the sender identity to detect man-in-middle attacks as requested. This is necessary for multicast protocols where SSL cannot be used. The goal of this procedure is to perform integrity check and to alert the user if any of the properties defined in Sect. 1 is violated by the received stream.

The client calculates its own hash chain analogously to the sender procedure Sect. 4.2 starting from the value F_0 received with SDP. HE_i^r is the hash chain element for the I-frame F_i^r received by the client. Therefore for each received frame the procedure compares HE_i^r with the received value HE_i if $HE_i^r \neq HE_i$ the frame is marked as tampered.

When the client receives a packet with the NUT carrying a signature DS_{HE_i}:

1. It retrieves the its own hash chain element HE_i^r corresponding to frame F_i.
2. It checks if HE_i^r is equals to the value HE_i used for digital signature.
3. It verifies the signature of the received hash chain element HE_i using the public key Pk_C

$$Verify(DS_{HE_i}, HE_i, Pk_C) \tag{3}$$

4. If the signature verification fails all the frames starting from the last signed I-frame that has been successfully verified up to the next signed I-frame that will pass the step (3) will be marked as untrusted or better potentially tampered.

If an attacker tampered an I-frame in stream. We can have two possible situations:

(a) The tampered frame is a signed I-frame.
(b) The tampered frame is not a signed I-frame.

In the first case, by the properties of the digital signature, the attacker is not able to produce a valid signature for the hash chain element HE_i^t for frame F_i^t because he do not know Sk_S, so the verification fails on step (3).

In the second case $F_i \neq F_i^r \Rightarrow HE_i \neq HE_i^r \Rightarrow HE_{i+1} \neq HE_{i+1}^r \Rightarrow \cdots \Rightarrow HE_j \neq HE_j^r$ by the properties of the hash chain, where j is the next signed I-frame. When the client will receive the signed frame F_j the verify phase will fail on step (2) and also the frame F_i^r will be marked as tampered by step (4).

Analogously is possible to demonstrate that if the attacker tampered a sequence of frame he is not able to forge the verification token. In-fact if we can conduct this problem to the following two cases:

1. The tampered sequence includes one or more signed I-frames, and this case is analogous to the case (a) in which is tampered a single signed I-frame and the attacker is not able to reproduce the digital signature.
2. The tampered sequence does not include signed I-frames. In this case, similarly to the case (b), the procedure Sect. 4.3 fails on step (2).

4.4 Restart on Lost Packet

We assumed to use RTSP over UDP that is a connectionless protocol. Therefore some packets may be lost and our method must hand such event.

For our method, the loss of a packet is equivalent to the deletion of a packet, that causes the failure of the procedure 4.3 by the client.

If the lost packets belong to P-frames or B-frames, nothing happens to the stream because those frames are not considered in our method.

Otherwise the lost packets belong to I-frames or signed I-frames. In such a case all the frames until the next frame with a valid signature DS_{HE_i} will be marked as untrusted and the procedure restarts the hash chain with the signed value of HE_i.

5 Implementation Details

In this section we will show some implementation details of the proposed method. We also will give some indication about the the cryptographic function we have chosen to use according to the recommendations of the National Institute of Standards and Technology (NIST) [4].

5.1 Setup Procedure

In Sect. 4.1 we described the setup procedure for our method. Due to the flexibility of SDP, this procedure has been straight embedded in the `DESCRIBE` command of RTSP. The *nonce* is produced an integer random number generator with a 1024 bit precision. Finally the strength of the entire method strictly depends on the robustness of the asymmetric signature algorithm and corresponding key pair used for. Following the recommendation of NIST [4] we adopted for the *RSA* algorithm with a 3072 bit key pair due to its wide availability. More efficient algorithms such as *ECDSA* could be used in order to save space and computational time. In this case the suggested key size by [4] is 192 bit.

5.2 Sender and Receiver Procedure

As far as the hash function concern we adopted the $SHA256$ function which as recommended by NIST [4]. Stronger hash functions like $SHA512$ (with a length of 512 bit), despite of their light computational cost, could produce a deeper impact on network usage as the result of this function is added to each I-frame.

It is important to highlight that the signature it is always a time consuming process. For this reason we designed the method to send signed I-frames is performed only at regular intervals of n instead of signing each frames. The value of n can be adapted to the resource available.

On the other hand the hash function computed for each I-frame, is light and can be even efficiently implemented in a dedicated hardware.

Obviously the implementation and the issues of the receiver procedure are specular to the sender procedure.

5.3 Implementation

In order to verify the feasibility of our method we engineered a real size case study.The server runs on a Raspberry Pi 3 model B with a 1.2 GHz 64-bit quad-core ARMv8 CPU equipped with a webcam module version 2 supported by Video for Linux (V4L) through the kernel module bcm2835-v4l2. The operating system installed on the server was Rasbian Version 8.0 (jessie). The server side application was based upon the framework live555 (http://www.live555.com/).

We performed several runs with different camera configurations, changing resolution and frame-rate. Despite of the low CPU performance, during all the tests the CPU usage was always under 50%, confirming that our method can be implemented even on small embedded CPU.

On the client side we used a standard linux virtual machine with Ubuntu Linux 14.04 with 1 GB RAM and a 2 Core Intel i7 CPU at 2.6 GHz. On the client side we used mplayer to reproduce the stream without checking the integrity and a custom client application to check the stream integrity before passing the frames to a standard player. Figure 1 shows the client output with the video on the right side and the execution trace on the left hand displaying the results of the hash chain verification.

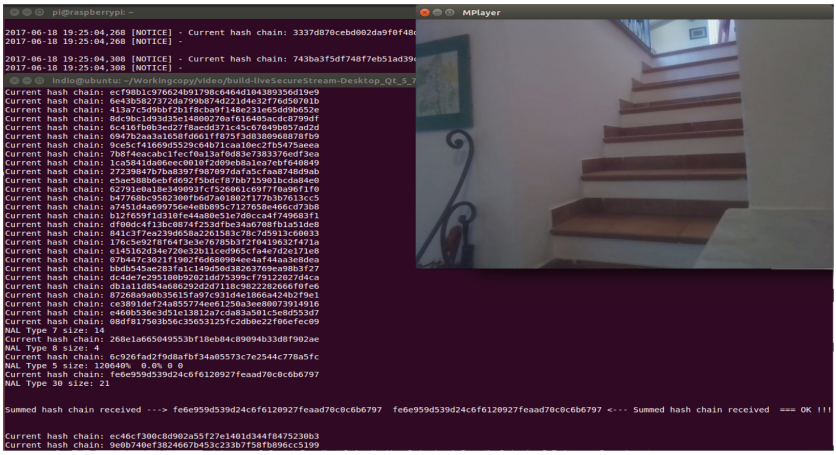

Fig. 1. Client: Example execution with hash verification and mplayer output

Currently we are working on a new version of the server implementation on an AXIS/ACAP platform.

6 Conclusions and Future Works

In Sect. 4 we showed a method that links each I-frame in a stream building an hash chain and while some I-frames are digitally signed. This permits a SRC to send a stream to a set of clients via multicast UDP protocol, preventing an attacher to tamper the stream by adding, modifying or deleting one ore more frames. This is an enhancement with respect to the standard SSL connection properties, because it works on multicast connections over a connectionless weaker protocol like UDP.

In addition the control information added to the original stream are sent only using an unspecified NUT, preserving therefore the compatibility with H.264. This means that the secure stream can be processed by a standard H.264 client. Even if the method has been conceived for live streams, these streams can also be stored in files and verified off-line with the same procedure Sect. 4.3.

The proposed method is robust to the loss of packets and a recovery procedure has been presented in Sect. 4.4 can be executed when packet are lost.

We implemented a prototype of a SRC using a Raspberry PI with a camera module in order to test this method in a real word scenario.

Actually we are planning to add a symmetric encryption algorithm to the stream to preserve privacy limiting the list of authorized client. In-fact only the client that participated to the setup phase, negotiating the session key will be able to decrypt the stream.

Further Information

Part of the work presented in this paper is protected by an Italian patent registered with number 102016000007162 on January 26th 2016 owned by eTuitus.

References

1. I.T.S.G.: Itu-t h.264 (v11): Advanced video coding for generic audiovisual services. ITU-T Series H: Audiovisual And Multimedia Systems October 2016. http://handle.itu.int/11.1002/1000/12904
2. I.T.S.G.: Itu-t H.264: Advanced video coding for generic audiovisual services. ITU-T Series H: Audiovisual And Multimedia Systems November 2007. http://handle.itu.int/11.1002/1000/9226
3. Albano, P., Bruno, A., Carpentieri, B., Castiglione, A., Castiglione, A., Palmieri, F., Pizzolante, R., You, I.: A Secure Distributed Video Surveillance System Based on Portable Devices. In: Quirchmayr, G., Basl, J., You, I., Xu, L., Weippl, E. (eds.) CD-ARES 2012. LNCS, vol. 7465, pp. 403–415. Springer, Heidelberg (2012). doi:10.1007/978-3-642-32498-7_30
4. Barker, E.: Recommendation for key management part 1: general (revision 4). NIST Spec. Publ. **800**(57), 1–160 (2016)
5. Bayram, S., Sencar, H.T., Memon, N., Avcibas, I.: Source camera identification based on CFA interpolation. In: IEEE International Conference on Image Processing, ICIP 2005, vol. 3, pp. III-69. IEEE (2005)

6. Bayram, S., Sencar, H.T., Memon, N.: Video copy detection based on source device characteristics: a complementary approach to content-based methods. In: Proceedings of the 1st ACM International Conference on Multimedia Information Retrieval, pp. 435–442. ACM (2008)
7. Bruno, A., et al.: Secure and distributed video surveillance via portable devices. J. Ambient Intell. Humanized Comput. **5**(2), 205–213 (2014). doi:10.1007/s12652-013-0181-z
8. Castiglione, A., Cepparulo, M., De Santis, A., Palmieri, F.: Towards a lawfully secure and privacy preserving video surveillance system. In: Buccafurri, F., Semeraro, G. (eds.) EC-Web 2010. LNBIP, vol. 61, pp. 73–84. Springer, Heidelberg (2010). doi:10.1007/978-3-642-15208-5_7
9. Cattaneo, G., Roscigno, G., Bruno, A.: Using PNU-based techniques to detect alien frames in videos. In: Blanc-Talon, J., Distante, C., Philips, W., Popescu, D., Scheunders, P. (eds.) ACIVS 2016. LNCS, vol. 10016, pp. 735–746. Springer, Cham (2016). doi:10.1007/978-3-319-48680-2_64
10. Chen, M., Fridrich, J., Goljan, M., Lukáš, J.: Source digital camcorder identification using sensor photo response non-uniformity. In: Electronic Imaging 2007, p. 65051G. International Society for Optics and Photonics (2007)
11. Fridrich, J., Lukáš, J., Goljan, M.: Digital camera identification from sensor noise. IEEE Trans. Inf. Secur. Forensics **1**(2), 205–214 (2006)
12. Goljan, M., Fridrich, J., Filler, T.: Large scale test of sensor fingerprint camera identification. In: IS&T/SPIE, Electronic Imaging, Security and Forensics of Multimedia Contents XI, vol. 7254, pp. 1–12. International Society for Optics and Photonics (2009)
13. Hsu, C.C., Hung, T.Y., Lin, C.W., Hsu, C.T.: Video forgery detection using correlation of noise residue. In: IEEE 10th Workshop on Multimedia Signal Processing, pp. 170–174. IEEE (2008)
14. Hyun, D.K., Lee, M.J., Ryu, S.J., Lee, H.Y., Lee, H.K.: Forgery detection for surveillance video. The Era of Interactive Media, pp. 25–36. Springer, New York (2013). doi:10.1007/978-1-4614-3501-3_3
15. Liao, S.Y., Huang, T.Q.: Video copy-move forgery detection and localization based on Tamura texture features. In: 2013 6th International Congress on Image and Signal Processing (CISP), vol. 2, pp. 864–868. IEEE (2013)
16. Ramaswamy, N., Rao, K.: Video authentication for H. 264/AVC using digital signature standard and secure hash algorithm. In: NOSSDAV 2006. Newport, Rhode Island, USA (2006)
17. Redfield, C.M., Date, H.: Gringotts: securing data for digital evidence. In: Security and Privacy Workshops (SPW), pp. 10–17. IEEE (2014)
18. Schneier, B.: Applied Cryptography: Protocols, Algorithms, and Source Code in C. Wiley, Hoboken (2007)
19. Schulzrinne, H., Rao, A., Lanphier, R.: Real time streaming protocol (RTSP). Internet Engineering Task Force. Technical report, RFC 2326 (1998)
20. Stütz, T., Uhl, A.: Format-compliant encryption of H. 264/AVC and SVC. In: Tenth IEEE International Symposium on Multimedia, ISM 2008, pp. 446-451. IEEE (2008)
21. Team, J.V.: Advanced video coding for generic audiovisual services. ITU-T Rec. H **264**, 14496–14510 (2003)
22. Yeung, S., Lui, J.C., Yau, D.K.: Secure real-time streaming protocol (RTSP) for hierarchical proxy caching. Int. J. Netw. Secur. **7**(3), 1–20 (2007)